

LL(k)-Grammatiken

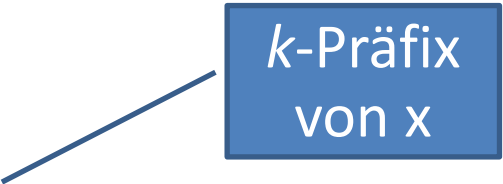
Sei $G = (N, T, P, S)$ eine kontextfreie Grammatik und k eine ganze Zahl.
 G ist eine **LL(k)-Grammatik**, wenn Folgendes gilt:

Falls es zwei **Linksableitungen**

$S \Rightarrow^* uY\alpha \Rightarrow u\beta\alpha \Rightarrow^* ux$ und

$S \Rightarrow^* uY\alpha \Rightarrow u\gamma\alpha \Rightarrow^* uy$ mit $k:x = k:y$

gibt, dann gilt $\beta = \gamma$.



k -Präfix
von x

Die Expansion des nächsten Nichtterminals ist immer eindeutig bestimmt durch die **bisher erkannten Zeichen/Token** und die **nächsten k Zeichen/Token** in der Eingabe.

Ist diese Grammatik LL(1) ?

S	$\rightarrow$	if id then S else S fi
	$ $	while id do S od
	$ $	begin S end
	$ $	id := id

Falls es zwei Linksableitungen

$S \Rightarrow^* uY\alpha \Rightarrow u\beta\alpha \Rightarrow^* ux$ und

$S \Rightarrow^* uY\alpha \Rightarrow u\gamma\alpha \Rightarrow^* uy$ mit $k:x = k:y$
gibt, dann gilt $\beta = \gamma$.

Betrachte jeden möglichen 1-Präfix $1:x = 1:y = z$

Also z.B. für $z = \text{if}$ folgt $\beta = \gamma = \text{if id then } S \text{ else } S \text{ fi}$

Für jeden 1-Präfix folgt $\beta = \gamma$, also ist die Grammatik LL(1).

Ist diese Grammatik LL(1) ?

S	$\rightarrow$	if id then S else S fi
		while id do S od
		begin S end
		id := id
		id (id)

Falls es zwei Linksableitungen

$S \Rightarrow^* uY\alpha \Rightarrow u\beta\alpha \Rightarrow^* ux$ und

$S \Rightarrow^* uY\alpha \Rightarrow u\gamma\alpha \Rightarrow^* uy$ mit $k:x = k:y$

gibt, dann gilt $\beta = \gamma$.

Betrachte zwei Ableitungen mit $\beta = \text{id} := \text{id}$ bzw. $\gamma = \text{id} (\text{id})$

Es ergibt sich zwangsläufig der 1-Präfix $1:x = 1:y = \text{id}$, obwohl $\beta \neq \gamma$.

Also ist die Grammatik *nicht* LL(1).

Die Grammatik ist aber LL(2), da die 2-Präfixe $2:x = \text{id} :=$ und $2:y = \text{id}($ verschieden sind.

Ist diese Grammatik LL(1) ?

S	$\rightarrow$	if id then S else S fi
		while id do S od
		begin S end
		$V := V$
		id (L)
V	$\rightarrow$	id id (L)
L	$\rightarrow$	id id, L

Falls es zwei Linksableitungen

$S \Rightarrow^* uY\alpha \Rightarrow u\beta\alpha \Rightarrow^* ux$ und

$S \Rightarrow^* uY\alpha \Rightarrow u\gamma\alpha \Rightarrow^* uy$ mit $k: x = k: y$
gibt, dann gilt $\beta = \gamma$.

Annahme: Es gibt ein k für das die Grammatik LL(k) ist. Betrachte zwei Ableitungen mit $\beta = V := V$ bzw. $\gamma = \text{id (} L \text{)}$ und $x = \underbrace{\text{id (id, id, ..., id)}}_{\lceil \frac{k}{2} \rceil \text{ mal}} := \text{id}$ sowie $y = \underbrace{\text{id (id, id, ..., id)}}_{\lceil \frac{k}{2} \rceil \text{ mal}}$

Die k -Präfixe sind gleich $k: x = k: y$, obwohl $\beta \neq \gamma$. Also ist die Grammatik *nicht* LL(k).

Eigenschaften von LL(k)-Grammatiken

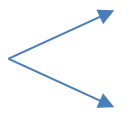
Falls es zwei Linksableitungen

$S \Rightarrow^* uY\alpha \Rightarrow u\beta\alpha \Rightarrow^* ux$ und

$S \Rightarrow^* uY\alpha \Rightarrow u\gamma\alpha \Rightarrow^* uy$ mit $k:x = k:y$ gibt, dann gilt $\beta = \gamma$.

- G ist linksrekursiv $\Rightarrow G$ ist **nicht** LL(k)

$S \rightarrow Sa \mid b$

$u\alpha = \varepsilon$  $S \Rightarrow^* Sx$ mit $x=ba \dots a$ (k-mal a)
 $S \Rightarrow^* Sy$ mit $y=xa \Rightarrow$ k-Präfix von y und x ist gleich

- G ist mehrdeutig $\Rightarrow G$ ist **nicht** LL(k)

Der erste Schritt, um eine LL(k)-Grammatik zu erhalten, besteht darin, sie **eindeutig zu machen** und **Linksrekursion zu entfernen**.

LL(1)-Parser

- LL(1) bedeutet, dass es für jedes Nichtterminal und jedes Token genau eine Produktion gibt.

Parser kann durch 2D Tabelle gesteuert werden:

- Eine Dimension für das zu expandierende Nichtterminal
- Eine Dimension für das nächste Token (Vorausschau)
- Jeder Tabelleneintrag besteht aus maximal einer Produktion.

	a	b	...
S	$S \rightarrow \alpha$	$S \rightarrow \beta$	...
A	×	$A \rightarrow \gamma$	...
...	...	...	...

$$S \Rightarrow^* uA\alpha \Rightarrow u\gamma\alpha$$

Generierung von Top-Down-Parsern

- Erzeuge Tabelle $M : N \times T \rightarrow P$
- $M[n, a]$ liefert die zu expandierende Produktion für das Nichtterminal n , wenn das aktuelle Symbol a ist.
- **Voraussetzung:** Die Grammatik ist LL(1).

First- und Follow-Mengen

$$\text{First}_1(\alpha) = \{a \in T \mid \exists \beta: \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\}$$

$$\text{Follow}_1(X) = \{a \in T \mid \exists \alpha, \beta: S \Rightarrow^* \alpha X a \beta\}$$

$\downarrow$
?
 ε

Es gibt Algorithmen zur Berechnung dieser Mengen. Wir werden die Mengen für einfache Grammatiken durch Nachdenken bestimmen.

Generierungsalgorithmus

- **Eingabe:** Grammatik G
- Für alle $(X \rightarrow \alpha) \in P$ und $a \in \text{First}_1(\alpha)$
 - Setze $M[X, a] = (X \rightarrow \alpha)$
 - Falls $\varepsilon \in \text{First}_1(\alpha)$ d. h. $\alpha \Rightarrow^* \varepsilon$
 - Für alle $b \in \text{Follow}_1(X)$
 - Setze $M[X, b] = (X \rightarrow \alpha)$
- Alle nicht gesetzten Einträge von M sind Fehlerfälle.
- Wird ein **Eintrag zum zweiten Mal** gesetzt, dann ist die Grammatik **nicht LL(1) !!!**

Beispiel

- **Eingabe:** Grammatik G
- Für alle $(X \rightarrow \alpha) \in P$ und $a \in \text{First}_1(\alpha)$
 - Setze $M[X, a] = (X \rightarrow \alpha)$
 - Falls $\varepsilon \in \text{First}_1(\alpha)$
 - Für alle $b \in \text{Follow}_1(X)$
 - Setze $M[X, b] = (X \rightarrow \alpha)$
- Alle nicht gesetzten Einträge von M sind Fehlerfälle.
- Wurde ein Eintrag zweimal gesetzt, dann ist die Grammatik nicht LL(1) !!!

$$\text{First}_1(\alpha) = \{a \in T \mid \exists \beta: \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\}$$

$$\text{Follow}_1(X) = \{a \in T \mid \exists \alpha, \beta: S \Rightarrow^* \alpha X a \beta\}$$

1	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
2	<i>Expr</i>	$\rightarrow$	<i>Term Expr'</i>
3	<i>Expr'</i>	$\rightarrow$	$+$ <i>Term Expr'</i>
4		$ $	$-$ <i>Term Expr'</i>
5		$ $	ε
6	<i>Term</i>	$\rightarrow$	<i>Factor Term'</i>
7	<i>Term'</i>	$\rightarrow$	$*$ <i>Factor Term'</i>
8		$ $	$/$ <i>Factor Term'</i>
9		$ $	ε
10	<i>Factor</i>	$\rightarrow$	<u>num</u>
11		$ $	<u>id</u>
12		$ $	$($ <i>Expr</i> $)$

		$First(\alpha)$	$Follow(X)$ falls $\varepsilon \in First_1(\alpha)$
1	$Goal$	$\rightarrow Expr \#$	
2	$Expr$	$\rightarrow Term Expr'$	
3	$Expr'$	$\rightarrow + Term Expr'$	
4		$ - Term Expr'$	
5		$ \varepsilon$	
6	$Term$	$\rightarrow Factor Term'$	
7	$Term'$	$\rightarrow * Factor Term'$	
8		$ / Factor Term'$	
9		$ \varepsilon$	
10	$Factor$	$\rightarrow \underline{num}$	
11		$ \underline{id}$	
12		$ (Expr)$	

$$First_1(\alpha) = \{a \in T \mid \exists \beta: \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\} \quad Follow_1(X) = \{a \in T \mid \exists \alpha, \beta: S \Rightarrow^* \alpha X a \beta\}$$

Für alle $(X \rightarrow \alpha) \in P$
und $a \in \text{First}_1(\alpha)$:

- Setze $M[X, a] = (X \rightarrow \alpha)$
- Falls $\varepsilon \in \text{First}_1(\alpha)$:
 - Für alle $b \in \text{Follow}_1(X)$:
 - Setze $M[X, b] = (X \rightarrow \alpha)$

			$\text{First}(\alpha)$	$\text{Follow}(X)$ falls $\varepsilon \in \text{First}_1(\alpha)$
1	<i>Goal</i>	$\rightarrow$	<i>Expr #</i>	<u>num</u> , <u>id</u> , (
2	<i>Expr</i>	$\rightarrow$	<i>Term Expr'</i>	<u>num</u> , <u>id</u> , (
3	<i>Expr'</i>	$\rightarrow$	<i>+ Term Expr'</i>	+
4			<i>- Term Expr'</i>	-
5			ε	ε # ,)
6	<i>Term</i>	$\rightarrow$	<i>Factor Term'</i>	<u>num</u> , <u>id</u> , (
7	<i>Term'</i>	$\rightarrow$	<i>* Factor Term'</i>	*
8			<i>/ Factor Term'</i>	/
9			ε	ε # ,), +, -
10	<i>Factor</i>	$\rightarrow$	<u>num</u>	<u>num</u>
11			<u>id</u>	<u>id</u>
12			<i>(Expr)</i>	(

LL(1)-Tabelle für Ausdrucksgrammatik

Beispiele:

$(x - 2) * y$ wird akzeptiert

$(x -) 2 * y$ wird *nicht* akzeptiert

	+	-	*	/	<u>num</u>	<u>id</u>	(	)	#
<i>Goal</i>	—	—	—	—	$G \rightarrow E$	$G \rightarrow E$	$G \rightarrow E$	—	—
<i>Expr</i>	—	—	—	—	$E \rightarrow TE'$	$E \rightarrow TE'$	$E \rightarrow TE'$	—	—
<i>Expr'</i>	$E' \rightarrow +TE'$	$E' \rightarrow -TE'$	—	—	—	—	—	$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
<i>Term</i>	—	—	—	—	$T \rightarrow FT'$	$T \rightarrow FT'$	$T \rightarrow FT'$	—	—
<i>Term'</i>	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	$T' \rightarrow /FT'$	—	—	—	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
<i>Factor</i>	—	—	—	—	$F \rightarrow \text{num}$	$F \rightarrow \text{id}$	$F \rightarrow (E)$	—	—

G $\Rightarrow$

LL-Parser Treiber

Kelleralphabet für $S \rightarrow T+T$

$[S \rightarrow .T+T]$

$[S \rightarrow T.+T]$

$[S \rightarrow T+.T]$

$[S \rightarrow T+T.]$

```
class Parser {  
    var nextSymbol : Symbol  
    var stack : Stack<ProdRule>  
    var matrix : Matrix<ProdRule>
```

```
    proc scan() { /* reads next input symbol into nextSymbol */ }  
    proc error(message : String) { /* issues error message and stops the parser */ }  
    proc accept() { /* terminates parser successfully */ }  
    proc reduce() { /* replaces  $[X \rightarrow \beta.Y\gamma][Y \rightarrow \alpha.]$  by  $[X \rightarrow \beta Y. \gamma]$  */ }  
    proc pop() { /* removes topmost prodRule from stack */ }  
    proc push(p : ProdRule) { /* pushes p onto stack */ }  
    proc replaceBy(p : ProdRule) { /* replaces topmost prodRule of stack by p */ }  
    proc parse()  
}
```



LL-Parser Treiber

```
proc parse() {  
    scan()  
    push([S' → .S])  
  
    while (true) {  
        case stack.top in  
            [X → β.ay]: if (nextSymbol == a) {  
                scan()  
                replaceBy([X → βa.γ])  
            } else { /* error */ }  
  
            [X → β.Yγ]: if (matrix[Y,nextSymbol] == (Y → α)) {  
                push([Y → .α])  
            } else { /* error */ }  
  
            [X → α.]: reduce()  
            [X → .ε]: replaceBy([X → ε.])  
  
            [S' → S.]: if (nextSymbol == "#") {  
                accept()  
            } else { /* error */ }  
        }  
    }  
}
```

```

proc parse() {
  scan()
  push([S' → .S])

```

```

while (true) {
  case stack.top in
    [X → β.αγ]: if (nextSymbol == a) {
      scan()
      replaceBy([X → βα.γ]) } else { /* error */ }

    [X → β.Yγ]: if (matrix[Y,nextSymbol] == (Y → α)) {
      push([Y → .α]) } else { /* error */ }

    [X → α.]: reduce()
    [X → .ε]: replaceBy([X → ε.])

    [S' → S.]: if (nextSymbol == "#") {
      accept() } else { /* error */ } } } }

```

LL-Parser Treiber

	+	-	*	/	<u>num</u>	<u>id</u>	(	)	#
<i>G</i>	—	—	—	—	$G \rightarrow E$	$G \rightarrow E$	$G \rightarrow E$	—	—
<i>E</i>	—	—	—	—	$E \rightarrow TE'$	$E \rightarrow TE'$	$E \rightarrow TE'$	—	—
<i>E'</i>	$E' \rightarrow +TE'$	$E' \rightarrow -TE'$	—	—	—	—	—	$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
<i>T</i>	—	—	—	—	$T \rightarrow FT'$	$T \rightarrow FT'$	$T \rightarrow FT'$	—	—
<i>T'</i>	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	$T' \rightarrow /FT'$	—	—	—	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
<i>F</i>	—	—	—	—	$F \rightarrow \text{num}$	$F \rightarrow \text{id}$	$F \rightarrow (E)$	—	—

LL(1)-Tabelle für Ausdrucksgrammatik

Beispiele:

$(x - 2) * y$ wird akzeptiert

$(x-) 2 * y$ wird *nicht* akzeptiert

	+	-	*	/	<u>num</u>	<u>id</u>	(	)	#
<i>G</i>	—	—	—	—	$G \rightarrow E$	$G \rightarrow E$	$G \rightarrow E$	—	—
<i>E</i>	—	—	—	—	$E \rightarrow TE'$	$E \rightarrow TE'$	$E \rightarrow TE'$	—	—
<i>E'</i>	$E' \rightarrow +TE'$	$E' \rightarrow -TE'$	—	—	—	—	—	$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
<i>T</i>	—	—	—	—	$T \rightarrow FT'$	$T \rightarrow FT'$	$T \rightarrow FT'$	—	—
<i>T'</i>	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	$T' \rightarrow /FT'$	—	—	—	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
<i>F</i>	—	—	—	—	$F \rightarrow \underline{\text{num}}$	$F \rightarrow \underline{\text{id}}$	$F \rightarrow (E)$	—	—

$\# [S \rightarrow .G] [G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow .FT'] [F \rightarrow .(E)]$

Kellerinhalt	Eingabe
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow .FT'] [F \rightarrow .\underline{id}]$	x - 2 #
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow .FT'] [F \rightarrow \underline{id}.]$	- 2 #
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow F.T'] [T' \rightarrow \varepsilon.]$	- 2 #
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow FT'.]$	- 2 #
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow .-TE']$	- 2 #
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -.TE']$	2 #
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -.TE'] [T \rightarrow .FT'] [F \rightarrow .\underline{num}]$	2 #
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -.TE'] [T \rightarrow .FT'] [F \rightarrow \underline{num}.]$	#
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -.TE'] [T \rightarrow F.T'] [T' \rightarrow \varepsilon.]$	#
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -.TE'] [T \rightarrow FT'.]$	#
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -T.E'] [E' \rightarrow \varepsilon.]$	#
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -TE'.]$	#
$\#[G \rightarrow .E] [E \rightarrow TE'.] [E' \rightarrow \varepsilon.]$	#
$\#[G \rightarrow E.]$	#

Rechnung des Parsers

Beispiel: x - 2

➡ x - 2 wird akzeptiert

Rechnung des Parsers

Beispiel: x -

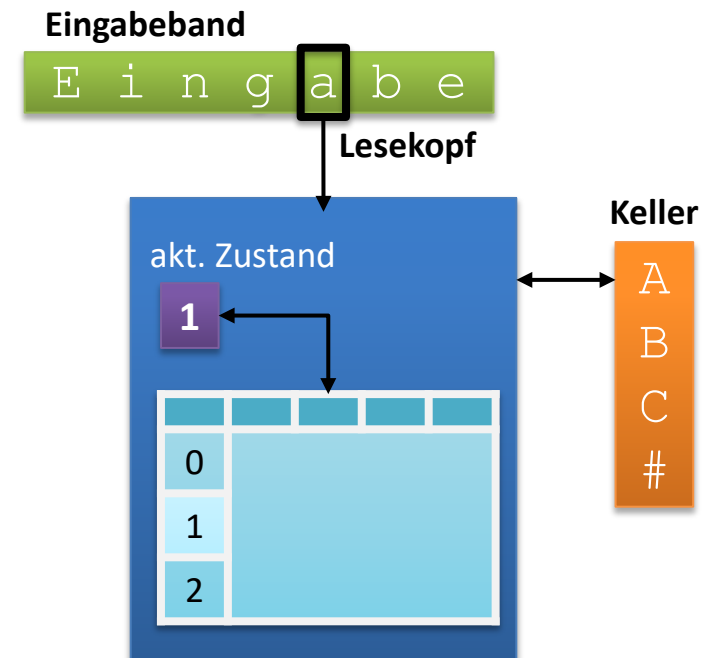
Kellerinhalt	Eingabe
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow .FT'] [F \rightarrow .\underline{id}]$	x - #
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow .FT'] [F \rightarrow \underline{id}.]$	- #
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow F.T'] [T' \rightarrow \varepsilon.]$	- #
$\#[G \rightarrow .E] [E \rightarrow .TE'] [T \rightarrow FT'.]$	- #
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow .-TE']$	- #
$\#[G \rightarrow .E] [E \rightarrow T.E'] [E' \rightarrow -.TE']$ error	#

➔ x - wird nicht akzeptiert, da keine Ableitungsregel der Form $T \rightarrow \varepsilon$ existiert

Kellerautomaten

Ein **nichtdeterministischer** Kellerautomat ist ein 7-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, F)$, wobei

- Z eine endliche Menge von **Zuständen**,
- Σ das **Eingabealphabet**,
- Γ das **Kelleralphabet**,
- $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Z \times \Gamma^*)$ die **Übergangsfunktion**, die nur auf *endliche* Teilmengen von $Z \times \Gamma^*$ abbildet,
- $z_0 \in Z$ der **Startzustand**,
- $\# \in \Gamma$ das **Anfangssymbol im Keller** und
- $F \subseteq Z$ eine Menge von **finalen Zuständen**



Beispiel: $\delta(q, \underline{\text{num}}, [E' \rightarrow +.TE']) = \{ (q', [E' \rightarrow +.TE'][T_ \rightarrow .FT']) \}$